

Nexus System — Algorithms & Machine Learning Techniques

Comprehensive Thesis-Ready Documentation

1. System Overview

Nexus is a **Local Retrieval-Augmented Generation (RAG)** research assistant that runs entirely in-browser. It allows users to upload PDF research papers and interact with them through an AI-powered chatbot. The system uses **no cloud APIs**—all AI inference, embedding generation, and semantic search happen on the user's device via **WebGPU** and **WebAssembly**.

The system's pipeline can be summarized as:

```
flowchart LR
    A["PDF Upload"] --> B["Text Extraction"]
    B --> C["Text Chunking"]
    C --> D["Embedding Generation"]
    D --> E["Vector Store (IndexedDB)"]
    F["User Query"] --> G["Query Embedding"]
    G --> H["Cosine Similarity Search"]
    E --> H
    H --> I["Context Retrieval"]
    I --> J["LLM Generation (Llama 3.2)"]
    J --> K["AI Response"]
```

2. Core Algorithms & Machine Learning Models

2.1 Large Language Model (LLM) — Transformer Architecture

Property	Value
Model	Llama-3.2-3B-Instruct-q4f16_1-MLC
Architecture	Decoder-only Transformer
Parameters	~3 Billion
Quantization	4-bit (q4f16_1)
Runtime	WebLLM (MLC AI) via WebGPU
Context Window	4,096 tokens
Sliding Window	2,048 tokens

Algorithm Description

The core generative intelligence of Nexus is **Meta's Llama 3.2-3B-Instruct**, a **decoder-only Transformer** model (Vaswani et al., 2017). This belongs to the family of **autoregressive language models** that generate text token-by-

token, each token conditioned on all previous tokens.

Key algorithmic components inside the Transformer:

- 1. **Scaled Dot-Product Self-Attention:** $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ Where Q (Query), K (Key), V (Value) are linear projections of the input, and d_k is the dimension of the key vectors. This mechanism allows each token to attend to all previous tokens in the sequence.
- 2. **Multi-Head Attention:** The model runs multiple attention heads in parallel, each learning different relationships: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$
- 3. **Grouped Query Attention (GQA):** Llama 3.2 uses GQA as an optimization where multiple query heads share a single key-value head, reducing memory and computation.
- 4. **RoPE (Rotary Positional Embedding):** Instead of absolute positional encodings, Llama uses rotary embeddings that encode relative position information directly into the attention computation.
- 5. **RMSNorm (Root Mean Square Normalization):** Used instead of LayerNorm for more efficient pre-normalization of each sub-layer.
- 6. **SwiGLU Activation Function:** A gated variant of the Swish activation used in the feed-forward layers: $\text{SwiGLU}(x, W, V, b, c) = \text{Swish}(xW + b) \otimes (xV + c)$
- 7. **4-bit Quantization (q4f16_1):** The model weights are quantized from 16-bit floating point to 4-bit integers using the MLC framework. This reduces model size by $\sim 4\times$ (from $\sim 6\text{GB}$ to $\sim 1.5\text{GB}$) while preserving most accuracy, enabling in-browser execution.
- 8. **Autoregressive Token Generation:** At inference time, the model generates one token at a time. Each new token is sampled from the probability distribution over the vocabulary, controlled by:
 - **Temperature** ($0.2-0.5$): Controls randomness. Lower values make output more deterministic.
 - **Top-k / Top-p sampling:** Limits the candidate set for the next token.

Relevant file: [useWebLLM.ts](#) — Model initialization, prompt engineering, and streaming inference.

2.2 Sentence Embedding Model — Sentence-BERT (SBERT)

Property	Value
Model	Xenova/all-MiniLM-L6-v2
Architecture	BERT-based Encoder Transformer
Parameters	~22 Million
Embedding Dimension	384
Runtime	Transformers.js (ONNX via WebAssembly)
Pooling Strategy	Mean Pooling

Algorithm Description

For semantic search, Nexus uses **all-MiniLM-L6-v2** (Wang et al., 2020; Reimers & Gurevych, 2019), a **Sentence-BERT (SBERT)** model. Unlike generative LLMs, this is an **encoder-only Transformer** whose purpose is to convert text into

dense vector representations (embeddings).

How it works:

1. **Tokenization:** Input text is tokenized into WordPiece subword tokens with a vocabulary of ~30,000 tokens.
2. **Encoder Layers:** The input passes through 6 Transformer encoder layers, each using:
 - Multi-head self-attention (12 heads)
 - Feed-forward networks
 - LayerNorm + Residual connections
3. **Mean Pooling:** After the final encoder layer, all token embeddings are averaged (mean-pooled) to produce a single 384-dimensional vector representing the entire input chunk:
$$\mathbf{e} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i$$
 where $\mathbf{h}_i$ is the hidden state of the i -th token.
4. **L2 Normalization:** The resulting embedding is normalized to unit length, which is critical for cosine similarity to work as a simple dot product.

Training Methodology: The model was fine-tuned using a **contrastive learning** objective on over 1 billion sentence pairs. Semantically similar sentences are pushed closer together in the embedding space, while dissimilar ones are pushed apart.

Relevant file: [vectorStore.ts](#) — Model selection and embedding pipeline initialization.

2.3 Cosine Similarity — Semantic Search Algorithm

Algorithm Description

Nexus uses **Cosine Similarity** as the core metric for ranking document relevance against a user query. This is the standard approach in **information retrieval** and **vector-based semantic search**.

Formula:
$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \times \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \times \sqrt{\sum_{i=1}^n b_i^2}}$$

Properties:

- Returns a value between **-1** and **1** (between 0 and 1 for normalized vectors).
- A score of **1** indicates identical direction (maximum semantic similarity).
- A score of **0** indicates orthogonality (no semantic relationship).
- Measures **angular distance** rather than magnitude, making it ideal for comparing semantic meaning regardless of text length.

How Nexus uses it:

1. The user query is embedded into a 384-dimensional vector.
2. This query vector is compared against every stored document chunk embedding using cosine similarity.
3. Results are sorted in descending order by similarity score.
4. The top-k results (default `k=3` for fast mode, `k=12` for chat context) are returned as relevant context.

Complexity: $O(n \times d)$ where n is the number of stored chunks and $d = 384$ is the embedding dimension (brute-force linear scan).

Relevant file: [vectorStore.ts](#) — The `cosineSimilarity()` private method implementation.

2.4 Text Chunking — Fixed-Size Sliding Window

Property	Value
Chunk Size	500 characters
Overlap	50 characters
Strategy	Fixed-size sliding window

Algorithm Description

Before documents can be embedded and searched, they must be split into manageable pieces. Nexus employs a **fixed-size sliding window chunking algorithm**:

```
Algorithm: FixedSizeChunking(text, chunkSize, overlap)
  chunks = []
  start = 0
  while start < len(text):
    end = min(start + chunkSize, len(text))
    chunks.append(text[start:end])
    start = start + chunkSize - overlap
  return chunks
```

Design rationale:

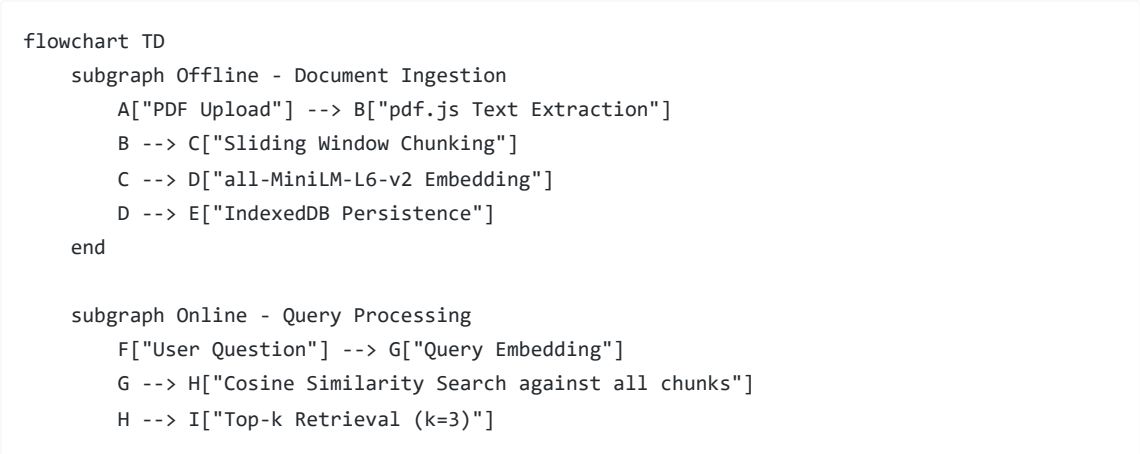
- **Chunk size (500 chars):** Balances between granularity (too small loses context) and specificity (too large dilutes meaning in the embedding).
- **Overlap (50 chars):** Ensures that information at chunk boundaries is not lost. A sentence split between two chunks will appear in both, maintaining coherence.
- **Character-based:** Uses raw character offsets rather than sentence or token boundaries for simplicity and predictability.

Relevant file: [vectorStore.ts](#) — The `chunkText()` private method.

2.5 Retrieval-Augmented Generation (RAG) Pipeline

Algorithm Description

RAG (Lewis et al., 2020) is the overarching paradigm that ties the system together. It combines **retrieval** (searching a knowledge base) with **generation** (producing natural language answers). Nexus implements a **complete local RAG pipeline**:



```
I --> J["Prompt Assembly: System Prompt + Context + Query"]
J --> K["Llama 3.2-3B Inference"]
K --> L["Streamed Response with Citations"]
end
```

Two-phase architecture:

Phase	Operation	Algorithm Used
Ingestion (Offline)	Parse PDF → Chunk text → Generate embeddings → Store	PDF.js extraction, Sliding Window Chunking, SBERT Embedding
Retrieval (Online)	Embed query → Search → Assemble prompt → Generate answer	SBERT Embedding, Cosine Similarity, Transformer LLM

Key design choice — In-context learning: Rather than fine-tuning the LLM on the user's documents, Nexus injects retrieved document excerpts directly into the prompt as context. The LLM then generates answers *grounded* in this context, following explicit instructions to cite sources and avoid hallucination.

Relevant files:

- [vectorStore.ts](#) — Ingestion and search
- [useWebLLM.ts](#) — RAG prompt assembly and generation
- [chatService.ts](#) — Context retrieval orchestration

2.6 Agentic Query Decomposition (Deep Research Mode)

Algorithm Description

Nexus implements a **Plan-Search-Synthesize** agent pattern (inspired by ReAct; Yao et al., 2022) in its **Deep Research** mode. This goes beyond standard single-pass RAG by using the LLM itself as a **planning agent**.

Algorithm steps:

```
Algorithm: DeepResearch(userQuery)
1. PLAN: Use LLM to decompose userQuery into 3 sub-queries
2. SEARCH: For each sub-query:
    results = vectorStore.search(sub_query, k=3)
    aggregate results into unified context
3. SYNTHESIZE: Use LLM to generate comprehensive report
    from aggregated context, with citations
return report
```

Phase 1 — Query Decomposition: The LLM receives the original complex query and is prompted to break it into 3 distinct, targeted sub-queries. This uses **structured output** (JSON format) to extract a machine-readable list:

- Temperature: **0.1** (very deterministic for reliable decomposition)
- Response format: `json_object`

Phase 2 — Parallel Retrieval: Each sub-query is independently searched against the vector store. Results are aggregated with source attributions.

Phase 3 — Report Synthesis: The aggregated context from all sub-queries is fed to the LLM with instructions to synthesize a structured, cited research report.

- Temperature: **0.5** (slightly more creative for report writing)

Relevant file: [DeepResearch.ts](#) — Complete agentic pipeline implementation.

2.7 Force-Directed Graph Layout Algorithm

Algorithm Description

The **Knowledge Graph** visualization uses a **force-directed graph layout**, implemented via the `react-force-graph-2d` library (based on D3-force). This is a **physics simulation algorithm** that positions nodes in 2D space.

Forces used:

Force	Description
Charge (Repulsion)	Nodes repel each other (Coulomb's law simulation) to prevent overlap
Link (Attraction)	Connected nodes are pulled together (spring force, Hooke's law)
Center	All nodes are attracted toward the center of the viewport
Collision	Nodes cannot overlap; treated as rigid circles

Algorithm (per simulation tick):

1. Calculate repulsive forces between all pairs of nodes: $O(n^2)$ or $O(n \log n)$ with Barnes-Hut optimization.
2. Calculate attractive forces along edges.
3. Update velocity and position of each node.
4. Apply damping to converge toward equilibrium.

Semantic links between documents are generated by using cosine similarity scores from the embedding space: when a new document is uploaded, the system searches for the most similar existing documents and creates graph edges labeled with match percentages.

Relevant files:

- [KnowledgeGraph.tsx](#) — Visualization component
- [page.tsx](#) — Semantic link calculation

2.8 Regex-Based Intent Detection (NLP Pattern Matching)

Algorithm Description

For file creation capabilities, Nexus uses **regular expression (regex) pattern matching**, a formal language-based approach for Natural Language Understanding (NLU):

- **Intent Detection:** A set of regex patterns (e.g., `/create\s+(?:a\s+)?file/i`) are tested against user messages to classify intent.
- **Entity Extraction:** Named entity extraction for filenames uses additional regex patterns to pull out file names and extensions.

This is a **rule-based NLP approach** as opposed to ML-based intent classification, chosen for its determinism and zero latency.

Relevant file: [fileCreationService.ts](#) — Pattern-based intent detection.

3. Summary Table — All Algorithms in Nexus

#	Algorithm / Technique	Category	Purpose in System	Key Reference
1	Transformer (Decoder-only)	Deep Learning / NLP	Natural language generation and understanding	Vaswani et al., 2017
2	Llama 3.2-3B-Instruct	Large Language Model	Core AI chatbot inference	Touvron et al., 2023 (Meta)
3	4-bit Quantization (q4f16)	Model Compression	Enables in-browser LLM execution	Dettmers et al., 2022
4	Sentence-BERT (all-MiniLM-L6-v2)	Sentence Embeddings	Converts text to 384-dim semantic vectors	Reimers & Gurevych, 2019
5	Mean Pooling	Feature Aggregation	Reduces per-token embeddings to per-sentence	Standard NLP practice
6	Cosine Similarity	Similarity Metric	Measures semantic relatedness between vectors	Salton & McGill, 1983
7	Fixed-Size Sliding Window Chunking	Text Processing	Splits documents into search-friendly pieces	IR best practice
8	Retrieval-Augmented Generation (RAG)	Information Retrieval + NLG	Grounds LLM answers in uploaded documents	Lewis et al., 2020
9	Agentic Query Decomposition	AI Agent Design	Breaks complex questions into sub-queries	Yao et al., 2022 (ReAct)
10	Force-Directed Graph Layout	Graph Visualization	Visualizes document relationships	Fruchterman & Reingold, 1991
11	WebGPU Accelerated Inference	Hardware Acceleration	GPU-accelerated model inference in browser	W3C WebGPU Spec
12	Regex Pattern Matching	Rule-based NLP	Intent detection for file creation	Thompson, 1968

4. Suggested Academic References

1. Vaswani, A., et al. (2017). "Attention Is All You Need." *NeurIPS*.
2. Touvron, H., et al. (2023). "LLaMA: Open and Efficient Foundation Language Models." *Meta AI*.
3. Reimers, N. & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *EMNLP*.
4. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*.
5. Wang, W., et al. (2020). "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." *NeurIPS*.
6. Dettmers, T., et al. (2022). "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale." *NeurIPS*.

7. Yao, S., et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR*.

8. Fruchterman, T.M.J. & Reingold, E.M. (1991). "Graph Drawing by Force-Directed Placement." *Software: Practice and Experience*.

9. Salton, G. & McGill, M.J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

10. Devlin, J., et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *NAACL*.

5. Thesis Chapter Suggestions

This content can support multiple thesis chapters:

Chapter	Content from This Document
Chapter 2: Review of Related Literature	Sections 2.1–2.8 (algorithm descriptions with academic citations)
Chapter 3: Methodology / System Design	Section 1 (system overview), Section 2.5 (RAG pipeline), Section 2.6 (Deep Research agent)
Chapter 4: System Architecture	The Mermaid diagrams and the summary table in Section 3
Chapter 5: Implementation	Code references and technical parameters throughout
Chapter 6: Results & Discussion	Section 2.3 (similarity scoring) and Section 2.6 (agent evaluation) provide metrics frameworks